

Encrypter and Decrypter of a AI Chatbot

Project Synopsis

Problem Statement

Conversations exchanged with an AI chatbot often contain personal or sensitive information. If these messages are stored or logged in plain, readable text, anyone who gains access to that storage — such as a hacker, or someone with physical access to the device — can read everything. A secure mechanism is required to protect chatbot conversation data from unauthorized access.

Objectives

Develop a cybersecurity application that securely encrypts and decrypts messages exchanged with an AI chatbot, ensuring that stored conversation data remains protected from unauthorized access.

Abstract

This project develops a secure AI chatbot application that encrypts and decrypts user messages using symmetric key encryption (via the Python cryptography library's Fernet module). The system accepts a message typed by the user, encrypts it before it is stored or logged, decrypts it only when it needs to be sent to the AI service to generate a reply, and encrypts the AI's response again before storage. It demonstrates cryptography, confidentiality, and secure handling of conversational data.

Methodology

Requirements analysis, environment setup (Python, VS Code, virtual environment), key generation, message encryption/decryption using the cryptography library, integration with the chatbot's message flow, testing with sample messages and invalid keys, validation and documentation.

Modules

1. Environment Setup (Python, VS Code, virtual environment)
2. Key Generation Module
3. Message Encryption Module
4. Message Decryption Module
5. Chatbot Integration (AI API communication)
6. Secure Storage / Logging
7. Error Handling (invalid key detection)

Technologies

Component	Technology
Language	Python
Libraries	cryptography (Fernet), os
IDE	VS Code
OS	Windows/Linux/Mac
Version Control	Git / GitHub

Working

1. User types a message.
2. Message gets encrypted (plaintext → ciphertext) using a secret key.
3. Encrypted message can be safely stored or logged.
4. Before sending to the AI, it is decrypted back (ciphertext → plaintext) using the same key.
5. The plain message is sent to the AI chatbot API.
6. The AI generates a response.
7. That response is also encrypted before being stored.
8. When shown to the user, it is decrypted again for display.

Simple Architecture

User → Types Message → Encryption Module (Fernet key) → Encrypted Message (stored securely) → Decryption Module → Plaintext sent to AI Chatbot API → AI Response → Encryption Module → Encrypted Response (stored) → Decryption Module → Response shown to User.

Key point: encryption protects the message while it is sitting in storage or in a log file. The moment it needs to go to the AI service to generate a reply, it must be decrypted, because AI models cannot process ciphertext directly.

Expected Outcome

Users can chat with the AI chatbot while their messages remain protected in storage. The project demonstrates cryptography, confidentiality, and secure handling of conversational data.

Future Scope

Password-based key derivation, two-factor authentication, digital signatures for message integrity, blockchain-based audit logs, multiuser support.